

Nutrition Plan Generator App — Final Design Decisions + Codex Build Prompt

1. Important Platform Clarification

This project is a **Flutter** mobile app.

Therefore, it **cannot be tested using Expo Go**, because Expo Go is for **React Native / Expo** apps, not Flutter apps.

For this Flutter project, the trial/testing guide should explain how to run the app using:

- Android physical device
- Android emulator
- iOS simulator, if on macOS
- iPhone physical device, if on macOS with Xcode
- `flutter run`

If the project is ever switched to React Native + Expo later, then Expo Go becomes relevant. For now, do **not** build this app as Expo/React Native. Build it as Flutter.

2. Product Summary

Build a **one-page nutrition plan generator app**.

The user enters body data, goal, meal preferences, and food restrictions. The app then generates:

- Daily calorie target
- Daily macro targets
- Meal-by-meal calorie and macro targets
- A meal plan
- Food quantities in grams
- Alternative food swap options

The MVP should work entirely on-device with no backend, no database, and no internet requirement.

3. Major Design Decisions Already Locked

3.1 Platform

Use:

Flutter

Reason:

- Android + iOS from one codebase
 - Stable cross-platform behavior
 - Good scalability for this app type
 - Fast development
 - Consistent UI
 - Strong performance when written cleanly
-

3.2 MVP Scope

Start with a **single-page app**.

No login. No user accounts. No trainer dashboard. No clients. No backend. No cloud sync. No database.

The single page should allow the user to:

1. Enter inputs
2. Press generate
3. View calories/macros
4. View generated meals
5. View food quantities
6. View alternatives/swaps

Later versions may add:

- User accounts
 - Trainer accounts
 - Multiple clients per trainer
 - Saved plans
 - Cloud sync
 - Backend API
 - Database
-

3.3 Data Source

Use a bundled local JSON file:

```
assets/data/foods.json
```

The JSON file is the app's local food dataset for launch.

Do not use PostgreSQL. Do not use Supabase. Do not use ORM. Do not use backend API for MVP.

The provided `foods.json` is the starting curated dataset. It will be expanded manually over time.

3.4 Scalability Decision

Even though the data source is JSON, the app must be architected cleanly so the data source can be replaced later.

Use a `FoodRepository` abstraction.

The UI and plan generator must **never read JSON directly**.

Correct flow now:

```
Flutter UI
→ PlanGenerator
→ FoodRepository
→ bundled foods.json
```

Possible future flow:

```
Flutter UI
→ PlanGenerator
→ FoodRepository
→ SQLite / Supabase / backend API
```

Only the repository implementation should change later, not the UI or nutrition engine.

3.5 Code Quality Decision

The app must be written like a professional project, not a quick messy prototype.

Requirements:

- Clean folder structure
- Clear separation of concerns
- No business logic inside UI widgets
- Models for structured data
- Repository for food access
- Services for nutrition calculations and generation logic
- Config/constants for formulas and rules
- Reusable widgets
- Maintainable naming
- Performance-conscious implementation

Performance requirements:

- Load `foods.json` once
- Parse it into Dart models
- Cache the parsed food list in memory
- Avoid repeated JSON parsing
- Avoid expensive repeated filtering where unnecessary
- Keep generation logic efficient enough for around 10k food rows
- Do not overload the user's device

3.6 UI Design Style

Use a premium minimal design inspired by:

Apple Health + modern nutrition dashboard + Liquid Glass / glassmorphism

Style requirements:

- Clean
- Premium
- Minimal
- Rounded cards
- Soft translucent/glass-like panels
- Smooth animations
- Simple colors
- No childish gym-bro design
- No clutter

- Mobile-first layout

Flutter can imitate Liquid Glass visually using custom UI, blur, opacity, rounded cards, and smooth transitions. Do not attempt to use real iOS Liquid Glass APIs in Flutter.

4. Food JSON Schema

The app should load foods from the provided JSON file.

Each food item has fields similar to:

```
{
  "id": "chicken_breast",
  "name": "Grilled Chicken Breast",
  "name_ar": "صدر فراخ مشوي",
  "macro_role": "protein",
  "calories_per_100g": 165,
  "protein_g_per_100g": 31.0,
  "carb_g_per_100g": 0.0,
  "fat_g_per_100g": 3.6,
  "is_vegan": false,
  "is_vegetarian": false,
  "allergens": [],
  "meal_tags": ["lunch", "dinner", "iftar"],
  "default_serving_g": 120
}
```

Do not require a database model or ORM.

Create a Dart model class such as:

Food

Fields:

- id
- name
- nameAr
- macroRole
- caloriesPer100g
- proteinGPer100g
- carbGPer100g
- fatGPer100g

- isVegan
- isVegetarian
- allergens
- mealTags
- defaultServingG

The app should validate required fields while loading.

5. User Input Fields

Use these as v1 input fields:

```
weight_kg
height_cm
body_fat_percentage
activity_level
goal
number_of_meals
number_of_snacks
diet_type
allergies
dislikes
milk_type
coffees_per_day
```

Do not require age/sex for v1.

The formula choices should be configurable so they can be changed later.

6. Calorie Calculation Rules

6.1 Primary Formula

Use **Katch-McArdle** when body fat percentage is available.

```
lean_body_mass = weight_kg × (1 - body_fat_percentage / 100)

BMR = 370 + (21.6 × lean_body_mass)

maintenance_calories = BMR × activity_multiplier
```

6.2 Activity Multipliers

```
sedentary: 1.2  
light: 1.375  
moderate: 1.55  
very_active: 1.725  
athlete: 1.9
```

6.3 Fallback Formula

If body fat percentage is missing, use a simple bodyweight multiplier:

```
maintenance_calories = weight_kg × activity_factor
```

Activity factors:

```
sedentary: 26  
light: 30  
moderate: 34  
very_active: 38  
athlete: 42
```

6.4 Goal Adjustment

```
maintain: maintenance  
lose_weight: maintenance - 500  
lose_weight_aggressive: maintenance - 1000  
gain_weight: maintenance + 300
```

Also keep this concept in the code comments/config:

```
7700 kcal ≈ 1 kg body fat
```

7. Macro Calculation Rules

Use body-weight-based protein/fat and carbs as remaining calories.

```
protein_g = weight_kg × 2.0  
fat_g = weight_kg × 0.6
```

Then:

```
protein_calories = protein_g × 4  
fat_calories = fat_g × 9  
remaining_calories = target_calories - protein_calories - fat_calories  
carbs_g = remaining_calories / 4
```

Macro calories:

```
protein = 4 kcal/g  
carbs = 4 kcal/g  
fat = 9 kcal/g
```

These numbers should be stored in a config/constants file, not scattered across UI code.

8. Meal Split Defaults

Use these v1 meal split rules.

```
2 meals: 50% / 50%  
3 meals: 30% / 40% / 30%  
4 meals: 25% / 25% / 25% / 25%  
5 meals: 20% / 25% / 25% / 20% / 10%
```

Snack rule:

```
1 snack = 10% of daily calories  
2 snacks = 15% total
```

Snacks take calories first, then the remaining calories are split across meals.

Ramadan mode:


```
iftar: 50%
snack: 20%
suhoor: 30%
```

Meal split rules should be configurable.

9. Generation Algorithm

Use the old system-design algorithm as v1.

9.1 Overall Flow

1. Read user inputs.
2. Calculate maintenance calories.
3. Apply goal adjustment.
4. Calculate protein, fat, and carbs.
5. Split daily calories/macros across meals/snacks.
6. Load foods through FoodRepository.
7. Filter foods based on user restrictions.
8. Generate meals.
9. Adjust portions.
10. Validate meal totals.
11. Add alternatives.
12. Display plan.

9.2 Food Filtering

Filter out foods that:

- Contain allergens selected by the user
- Match disliked foods
- Are not vegan when vegan diet is selected
- Are not vegetarian when vegetarian diet is selected
- Do not match the current meal tag when meal tags are needed

Split allowed foods into:

```
proteinFoods
carbFoods
```

fatFoods
mixedFoods

based on `macro_role`.

9.3 Meal Generation

For each meal:

1. Pick one protein food.
 2. Pick one carb food.
 3. Pick one fat food if needed.
 4. Optionally add mixed/vegetable foods if suitable.
 5. Start with each food's `default_serving_g`.
 6. Calculate meal totals.
 7. Adjust portions toward target.
-

9.4 Portion Adjustment

Start with default portions.

Then adjust:

- If protein is too low, increase protein food.
- If protein is too high, decrease protein food.
- If calories are too low, increase carb food.
- If calories are too high, decrease according to adjustment logic.
- If fat is too low, increase fat food.
- If fat is too high, decrease fat food.

Use a maximum iteration count, for example 20 loops, to avoid infinite loops.

After adjustment, round portions to practical values:

nearest 5g or 10g

9.5 Tolerance Rules

Accept a meal if it is within these tolerances:

```
calories: ±15%  
protein: ±10g  
carbs: ±15g  
fat: ±10g
```

If a meal fails tolerance:

- Try different foods.
- Retry a limited number of times.
- If still failing, return the closest reasonable result and mark it as approximate.

9.6 Alternatives

For each main food item, provide up to 2 alternatives.

Alternative rules:

- Same macro role when possible
- Same meal tag when possible
- Must respect allergies/dislikes/diet type
- Similar macro profile when possible

For v1, same-quantity alternatives are acceptable.

Example:

```
120g chicken breast  
alternatives:  
- 120g turkey breast  
- 120g white fish
```

9.7 Swap/Rebalance Rule

When the user swaps a food item later:

1. Replace it with an allowed alternative.
2. Recalculate meal totals.
3. If the meal no longer fits, rebalance portions.

Preservation priority:

Carbs are highest priority and should change last.
Fats are medium priority.
Protein is lowest priority and should change first.

Adjustment order:

1. Adjust protein first
2. Adjust fats second
3. Adjust carbs last

10. Required Flutter Architecture

Create a clean architecture similar to this:

```
lib/  
  main.dart  
  app.dart  
  
  core/  
    constants/  
      nutrition_constants.dart  
      meal_split_config.dart  
      app_theme.dart  
    utils/  
      number_rounding.dart  
      macro_math.dart  
    errors/  
      app_exception.dart  
  
  data/  
    datasources/  
      local_food_data_source.dart  
    repositories/  
      food_repository_impl.dart  
  
  domain/  
    models/  
      food.dart  
      user_input.dart  
      nutrition_targets.dart  
      meal.dart  
      meal_item.dart
```

```
    generated_plan.dart
repositories/
  food_repository.dart
services/
  nutrition_calculator.dart
  meal_splitter.dart
  plan_generator.dart
  food_swap_service.dart

presentation/
  screens/
    home_screen.dart
  widgets/
    glass_card.dart
    input_section.dart
    goal_selector.dart
    preferences_section.dart
    macro_summary_card.dart
    meal_card.dart
    food_item_tile.dart
    generate_button.dart

assets/
  data/
    foods.json
```

11. Folder Architecture Explanation Document Requirement

Generate a separate Markdown file:

```
PROJECT_ARCHITECTURE.md
```

This file must explain:

1. The full folder structure.
2. What each folder is responsible for.
3. What each important file does.
4. How data flows through the app.
5. How `foods.json` is loaded.
6. How `FoodRepository` abstracts the data source.
7. How `NutritionCalculator` works.

8. How `PlanGenerator` works.
9. How the UI consumes generated plans.
10. How future migration to SQLite/API/backend would work.

The goal is that in later chats, someone can read `PROJECT_ARCHITECTURE.md` and understand the project without inspecting the whole codebase from scratch.

12. Flutter Trial / Running Guide Requirement

Because this is Flutter, do **not** write an Expo Go guide as the main guide.

Instead, create a Markdown guide:

```
RUNNING_THE_APP.md
```

It must explain how to run the Flutter app on:

Android physical device

Include steps:

1. Install Flutter SDK.
2. Install Android Studio or Android SDK tools.
3. Enable Developer Options on Android phone.
4. Enable USB debugging.
5. Connect phone by USB.
6. Run:

```
flutter doctor
flutter devices
flutter pub get
flutter run
```

Android emulator

Include steps:

1. Install Android Studio.
2. Create emulator.
3. Start emulator.
4. Run:

```
flutter devices
flutter run
```

iOS simulator

Mention:

- Requires macOS
- Requires Xcode

Commands:

```
open -a Simulator
flutter devices
flutter run
```

iPhone physical device

Mention:

- Requires macOS
- Requires Xcode
- Requires Apple developer setup/signing

Also include a small note:

```
Expo Go is not applicable because this project is Flutter, not React Native/
Expo.
```

13. Required App Behavior

When the app starts:

1. Load `foods.json` from assets.
2. Parse it once.
3. Cache foods in memory through `FoodRepository`.
4. Show the one-page input UI.

When user presses Generate:

1. Validate inputs.

2. Calculate target calories/macros.
 3. Split targets across meals/snacks.
 4. Generate plan using local foods.
 5. Display macro summary and meal cards.
-

14. Required Output on Screen

Show:

- Daily calories
 - Daily protein/carbs/fats
 - Meal cards
 - Meal target calories
 - Foods inside each meal
 - Quantity in grams
 - Calories/macros per meal
 - Alternatives per food item
-

15. Codex Task

Build the complete Flutter MVP app using the decisions above.

Requirements:

1. Create the Flutter project structure.
 2. Add `assets/data/foods.json` support.
 3. Implement Dart models.
 4. Implement `FoodRepository` abstraction and local JSON implementation.
 5. Implement nutrition calculation services.
 6. Implement meal split service.
 7. Implement plan generation service using the old system-design algorithm.
 8. Implement alternatives/swaps logic.
 9. Implement clean one-page UI.
 10. Implement premium minimal glass-inspired styling.
 11. Write `PROJECT_ARCHITECTURE.md`.
 12. Write `RUNNING_THE_APP.md`.
 13. Ensure the app runs on Android/iOS through Flutter.
 14. Do not use backend, database, ORM, Supabase, Firebase, or Expo.
 15. Do not put business logic directly inside UI widgets.
-

16. Final Reminder

This is not a disposable prototype. It is an MVP with a scalable architecture.

The app should be simple now, but professionally structured so that later we can add:

- SQLite
- Backend API
- Cloud database
- Users
- Trainers
- Clients
- Saved plans
- Remote food updates

without rewriting the core app logic.